MONASH University

# Field Guide to Processors, Datapaths and Instruction Execution



**AI & Generative AI tools may be used SELECTIVELY within this assessment.**

In this task, AI can be used as specified for one or more parts of the assessment task as described below.
As a reminder, you can use generative artificial intelligence (AI) in order to troubleshoot your solutions, explain concepts and assist you in preparing your reports. Both students in a group must fully understand the work submitted for assessment - if you do not understand your work, you may receive a zero interview score, which will result in a zero for the project.

Any use of generative AI for this purpose must be appropriately acknowledged, if you do not acknowledge your use of AI, it may be submitted as plagiarism under the academic integrity guidelines. You must not use generative artificial intelligence (AI) to generate any HDL for submission.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

1. represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
2. adhere to the guidelines for AI use set for the assessment task.
3. reflect the University's commitment to academic integrity and ethical behaviour.

*Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.*

This lesson introduces some key background information for the project.

# Components of a Processor

## Introduction

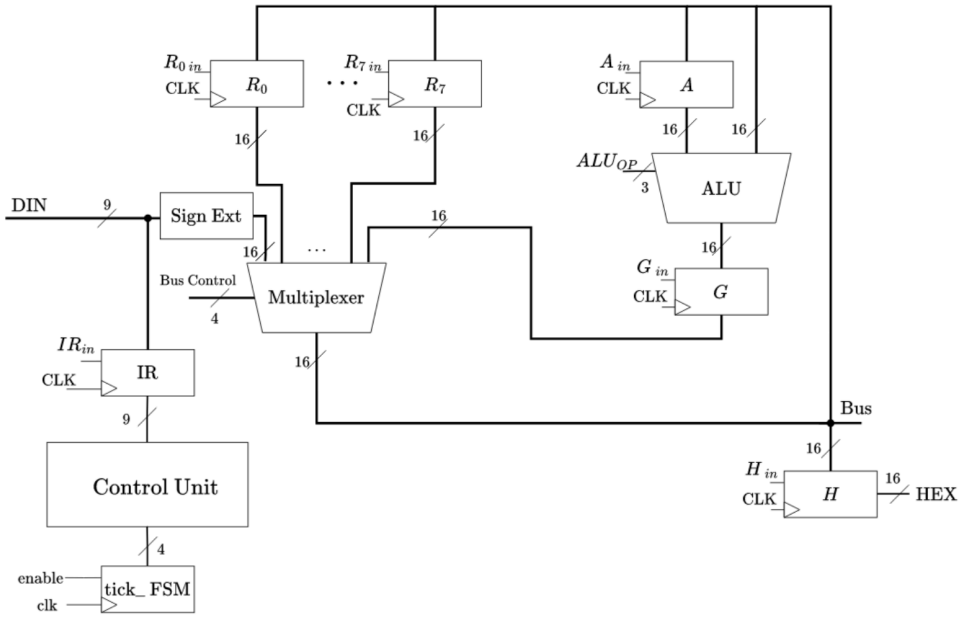
In this lesson we will introduce you to the key concepts behind the processor architecture that you will be creating in the project, which we have dubbed the x72 architecture. Many of the base components are simple building blocks (counters, registers, adders) that we have used independently in the unit.

You may recognise some of the concepts presented in this lesson from other units previous studies, or personal interests. However, this document is written without any assumptions about familiarity, beyond what we've taught in ECE2072. If you already know some of this, you are welcome to skip those sections.

## Components of a Processor

A computer processor, also known as a central processing unit (CPU), is a sequential digital circuit that performs the calculations that make a computer 'run'. It performs arithmetic and logical operations, input/output (I/O), and other basic instructions. In this project, you will be making your own processor. We're going to introduce you to a simple processor that only does some of these functions, and progressively add additional features and functionalities to build up to the x72 architecture you'll build in the project.

A simple processor has several individual components that all work together to perform operations, the processor that is designed in the project is shown in the image below.




## The ALU

The arithmetic logic unit (ALU) is a major component of the processor. It is an extension of some of the circuits you explored in lab 2, and performs all processes related to operations that the processor is capable of. Examples of these operations include arithmetic operations such as addition, subtraction, multiplication, and comparisons, as well as logical operations such as NOT, AND, and OR.

## The registers

We are familiar with registers (flip-flops, memory storage) from our study of sequential digital logic. In computer architecture, we reuse the term register for a fast, quickly available memory storage location to store values being actively operated on by the processor (in contrast to a bulk memory, i.e. RAM or ROM). A “register” in this context is just a bank of registers (D flip-flops) of the width of the data bus (ie 16-bits in our case).

Computer registers are divided into two categories: general purpose, which can store any value for use in programs, as well as some special purpose registers. An example of a special register in our processor is the “instruction register”, which stores the current instruction being executed.

## The bus

The bus is a central communications channel, of a width of 16 bits in our case, that connects most components of the processor. The bus needs to read from and write to many circuits. This is achieved with control signals and a multiplexer. The bus ensures that data can be passed between all necessary components to perform relevant computations by connecting various components within the processor.

## The control unit

The control unit is responsible for the retrieval, decoding, and execution of instructions. It achieves this by asserting various control signals, which turn on (enable) each of the individual logic units such as the registers, the ALU, and IO devices (for example, HEX displays). To take an example, the control signal R3in, tells register R3 to store whatever is on the bus.
The tick counter

How do we keep track of what signals the control unit needs to send? We need some sense of time, which is provided by a FSM. In our architecture, we have a small counter FSM that controls how far “through” executing an instruction our system is - in other words, it tells the control unit which part of a given instruction should be executed next. We will come back to this block later.

# Glossary

### Unsigned Integer:

A simple representation of a number in binary that can have a value from 0 to 2^N-1 where N is the number of bits in the number.

### Signed Integer:

A representation of a number (in binary) that can be positive or negative. In our processor, we are using 2's complement encoding to represent signed integers.

### Sign-Extension:

When data-buses don't have the same width, we need to do something to fill in the space. This is called extension. At a first glance this may not particularly seem like a big issue, as we could just add 0s to the most significant bits until our numbers are the same length, but that only works for strictly positive numbers! Since we are using 2s complement signed integers in our processor, we need to be able to pad the most significant bits with the appropriate values.

### Immediate:

The term used to refer to the region (range of bits) in an instruction where we store a binary number we wish to use as a value in our processor. The use of immediate values is how we load values into our processor.

### "Shift", "Bit Shift", "ssi"":

In binary arithmetic, a "shift" refers to shifting all of the bits in a number (often called a word) in a certain direction; this action is very similar to the behaviour of shift registers. To represent a bit shift operation, we use the symbol << and >> for left and right, respectively.

In our processor, we have combined both shift operations into a single instruction that uses a signed immediate to determine the direction of the operation.

The instruction 'ssi' refers to a logical "shift signed immediate" instruction. If the provided immediate is positive, you should perform a **left shift** by the number of bits in the immediate. If the immediate is negative, you should perform a **right shift** by the number of bits in the immediate. When you shift in a direction, you should introduce zeroes into the word to fill in the gaps.

Examples:

0010 << 1 = 0100

1001 >> 2 = 0010

Next Page

MONASH University

# Field Guide to Processors, Datapaths and Instruction Execution



**AI & Generative AI tools may be used SELECTIVELY within this assessment.**

In this task, AI can be used as specified for one or more parts of the assessment task as described below.
As a reminder, you can use generative artificial intelligence (AI) in order to troubleshoot your solutions, explain concepts and assist you in preparing your reports. Both students in a group must fully understand the work submitted for assessment - if you do not understand your work, you may receive a zero interview score, which will result in a zero for the project.

Any use of generative AI for this purpose must be appropriately acknowledged, if you do not acknowledge your use of AI, it may be submitted as plagiarism under the academic integrity guidelines. You must not use generative artificial intelligence (AI) to generate any HDL for submission.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

1. represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
2. adhere to the guidelines for AI use set for the assessment task.
3. reflect the University's commitment to academic integrity and ethical behaviour.

*Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.*

This lesson introduces some key background information for the project.

# Specification of x72 Processor

## Specification of x72 Processor

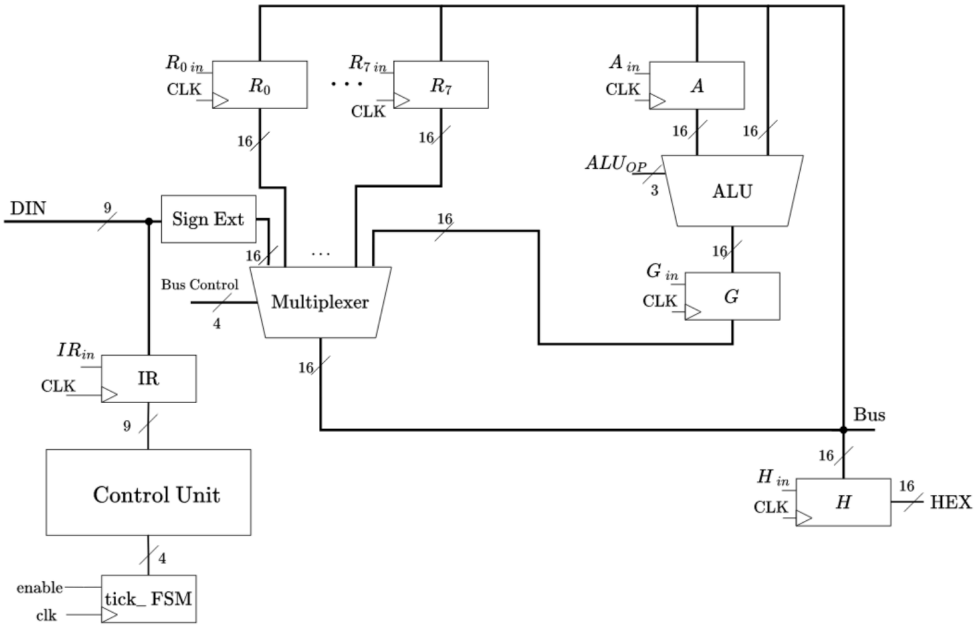
### Instructions

In order to tell the processor what it should be doing, we feed it a series of commands, referred to as "instructions". These instructions contain all of the information that the processor needs in order to execute each of its available operations (aka actions).

In our processor, instructions are received from the Din line (which will be connected to switches, or testbench inputs), and then clocked into the IR register so that the control unit can read them. For this processor, an instruction contains 9-bits, with a specific format that will be explained in the instruction encoding section, below.

### The Datapath

"Datapath" is the term for the collection of the components discussed above, arranged in a configuration that allows data to flow between them. The simplest design for a datapath uses one common bus that all components can write data to, or read data from. A simplified diagram of the x72 datapath is shown in the figure below.




The figure above shows a simple processor with some of the input and output signals being removed for clarity. The full list of inputs/outputs for each component are specified in Task 1.

Briefly:

- The DIN wires are on the middle left of the figure
- The ALU is on the upper right of the figure. It has two 16-bit data inputs, as well as an $ALU_{op}$ control signal input, which controls the operation that the ALU should perform. The A register holds an intermediate value for use in computation by the ALU, and the G register holds the result of the ALU's computation.
- There are 8 general purpose (GP) registers, $R_0$ to $R_7$ which store 16-bit data values. Note the 6 registers ($R_1$, $R_2$, ..., $R_6$) have been omitted from this diagram for simplicity.
- The IR register is a special-purpose 9-bit register that contains the current instruction.
- The (central) data bus has been implemented via a multiplexer. The multiplexer has 10 data inputs:
  - Value from the IR (current instruction)
  - Values from each GP register
  - Output from the ALU
- The select inputs of the multiplexer are tied to a wire called "Bus Control". We use this wire to specify which value should appear on the bus.
- The control unit has two inputs: the current instruction, as well as the tick FSM output:
  - The control unit decodes these inputs to set the ALUop, the Bus Control, and other control signals such as the Rin wires, Ain, Gin, ALUop, and the bus control lines (all of which are not shown in the figure).
- The tick FSM is required because each instruction takes more than one clock cycle to complete - each "instruction" is performed by executing a series of small steps, and at each step, a different set of control signals will be asserted.
- The reset signal is a synchronous signal that will reset all the register contents, including the instruction register, which holds the encoding of the instruction currently being executed.

## Operation Codes

Operation codes (opcodes) correspond to the different operations that are available. In this simple processor, you will use the most significant 3 bits of an instruction to specify the opcode, which means your processor can have up to 8 different instructions. The different possible opcode values correspond to individual instructions defined in the table below.

## Resetting the processor

If the reset input is asserted, nothing should happen until the next positive edge transition of the clock. At this point, the state of the CPU should be reset back to the initial state. This means that the contents of **all** registers should be set to 0, and the tick FSM should also be set to the 0th state. The x72 processor **must** use a synchronous reset.

## Instruction encoding

Now, let's talk about the instructions we want to have in our x72 architecture! All of our instructions specify the action to perform (what to do) in the most-significant three bits of the instruction, known as the **opcode**. A table of opcodes is provided below.

Note: Each task of the project will require you to implement different instructions from this page, so make sure that you read the project specification carefully to avoid unnecessary work at each step.

For each instruction (pattern of 9 bits), we need to encode two or three pieces of information:

- What we want the instruction to do (opcode)
- What values the instruction should operate on (e.g., values in one or two specific registers)
- What value we want the instruction to use (called an "immediate")

To make our lives easier, let's group instructions with similar requirements together. For this processor, there are two such types:

- Register to register operations (type 1):
  - These instructions move data that is already in the processor from one register to another, while transforming, combining, or modifying it in some manner.
  - These contain an opcode, and two register identifiers (addresses) labelled Rx and Ry.
- "Immediate" value operations (type 2):
  - These instructions introduce new information from the user into the register, also possibly transforming the data within the registers.
  - These contain an opcode, one register address (Rx), and an immediate value.

| Opcode | Operation | Function Performed | Description | Type |
|---|---|---|---|---|
| 0 | `disp Rx` | HEX = Rx | Displays a register's content on HEX as a decimal value | 2 |
| 1 | `add Rx, Ry` | Rx = Rx + Ry | Adds two register values | 1 |
| 2 | `addi Rx, Immi` | Rx = Rx + Immediate Value | Adds a register and an immediate value | 2 |
| 3 | `sub Rx, Ry` | Rx = Rx - Ry | Subtracts two register values | 1 |
| 4 | `mul Rx, Ry` | Rx = Rx * Ry | Multiplies two register values | 1 |
| 5 | `ssi Rx, Immi` | Rx = Rx << Immediate Value | Shifts Rx by the immediate value, +ve is left, -ve is right | 2 |
| 6 | `bez Rx, Immi` | If (Rx == 0) then PC = PC + 2 + BranchVal | If Rx is a zero, then add the appropriate branch val, calculated from Immi, to PC | 2 |
| 7 | `movi Rx, Immi` | Rx = Immediate Value | Moves an immediate value into a register | 2 |

**Note:** The branch instruction is only added as an extension during task 4, you can ignore it if you aren't going to attempt task 4.

### Type 1: Simple Register to Register Instructions

The add, sub, and mul instructions all work with two registers, and thus are classified as simple register-to-register instructions. In the x72 processor, we encode these as follows (remember, we use DIN to load the instruction into the instruction register (IR)):

| OPCODE | Rx | Ry |
|---|---|---|
| DIN[8:6] | DIN[5:3] | DIN[2:0] |

Registers are identified (or addressed) by their number as an **unsigned 3-bit binary number**: R0 can be addressed as 000, R3 as 011, etc.

Let's think about what an example of this type of instruction might look like in binary, using this formatting information, and the opcodes in the section above. For our example, let's consider the case of wanting to subtract the contents of R6 from the contents of R4. This can be written as:

```
sub R4, R6
```

That then breaks down as follows:

The opcode is 3, so Instruction[8:6] = 3'b011

Rx is 4, so Instruction[5:3] = 3'b100

Ry is 6, so Instruction[2:0] = 3'b110

In binary, the instruction will be: `Instruction[8:0] = 9'b011 100 110`

## Type 2: Separate Immediate Instructions:

As we are using only 9-bit instructions, we do not have room (in the way we would for larger 16 or 32-bit instructions) to communicate large immediate values to the processor inside a single instruction.

Therefore, the instructions disp, addi, movi and ssi are encoded differently using two ticks, as the immediate value must be provided separately.

For separate immediate instructions, only the opcode and Rx are provided in the main instruction, with the least significant three bits remaining unused. If required, the immediate value (needed for addi and movi) is provided to DIN on the clock tick ***after*** the initial decode tick. For disp, the instruction ends after the first 9 bits.

| OPCODE | Rx | Unused |
|---|---|---|
| CurrentDIN[8:6] | CurrentDIN[5:3] | CurrentDIN[2:0] |
| **Immediate** | | |
| NextDIN[8:0] | | |

Let's look at an example of this type of instruction. Let's say we want to add the number 15 to the current value of register R5. That could be written as:

```
addi R5, 15:
```

That will be broken down as follows:

The opcode is 2, so Instruction[8:6] = 3'b010

Rx is 5, so Instruction[5:3] = 3'b101

The bottom three bits are unused, so Instruction[2:0] = 3'bxxx

Note: the x here indicates a 'don't care'

We then need to supply the immediate value of 15 (Immi=9'd15, or 9'b000001111) onto DIN on the next tick.

Therefore overall, the instruction is:

Instruction[8:0] = 9'b010 101 xxx

DIN = 9'b000001111

The key idea here is that the immediate value is provided to the CPU input DIN wires at the next tick. Some more examples of this are provided below in the following section.

## Detailed breakdown of instructions

Inside the x72 processor, we want to split the execution of each instruction into 4-time stages called "ticks".

To keep things simple, we will artificially make any instruction that takes less than 4 ticks take the full 4 ticks by choosing to have the processor stay idle for any 'spare' ticks. During each tick, we will send control signals to the various parts of our processor in order to complete the required task. In general, each tick has the following purpose (this may be modified slightly if required):

Tick 1:

a. Read DIN into the instruction register so that the control unit can decode and process the instruction.

Tick 2:

a. Select the first piece of data to place onto the Bus (Bus control).
b. OR for ALU instructions, select the register (or immediate) to write onto the bus (General purpose registers Rx, ALU input register A, HEX output register H)

Tick 3:

a. For instructions that do not require four ticks, idle for this tick
b. If using the ALU, place the second input onto the Bus (Bus control)
c. Assert required ALU control signal to set the required operation
d. Assert ALU output register control wire Gin to store the ALU result

Tick 4:

a. For instructions that do not require four ticks, idle for this tick
b. Read the data from the ALU output register G onto the main bus (Bus control)
c. Select the register to read the bus data into (General purpose registers Rx)

**Previous Page** **Next Page**

MONASH University

# Field Guide to Processors, Datapaths and Instruction Execution



**AI & Generative AI tools may be used SELECTIVELY within this assessment.**

In this task, AI can be used as specified for one or more parts of the assessment task as described below.
As a reminder, you can use generative artificial intelligence (AI) in order to troubleshoot your solutions, explain concepts and assist you in preparing your reports. Both students in a group must fully understand the work submitted for assessment - if you do not understand your work, you may receive a zero interview score, which will result in a zero for the project.

Any use of generative AI for this purpose must be appropriately acknowledged, if you do not acknowledge your use of AI, it may be submitted as plagiarism under the academic integrity guidelines. You must not use generative artificial intelligence (AI) to generate any HDL for submission.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

1. represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
2. adhere to the guidelines for AI use set for the assessment task.
3. reflect the University's commitment to academic integrity and ethical behaviour.

*Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.*

This lesson introduces some key background information for the project.

# Tick by Tick Description of Each Instruction

## Tick by Tick Description of Each Instruction

The steps to be undertaken at each tick for each operation are enumerated below, except for branch which is explained in the next section. Remember that instructions are implemented progressively across the processor, so please check the project specification.

### disp Rx

Suppose Rx contained the value of 45. The instruction disp Rx will result in the following actions:

Tick 1:

Read DIN into the instruction register so that the control unit can decode and process the instruction.

Tick 2:

The value from Rx (45) is clocked into the HEX. HEX4 to HEX0 will display the decimal value of HEX from this tick (HEX4, HEX3, HEX2 will display "0". HEX1 displaying "4", HEX0 displaying "5").

Tick 3 & Tick 4:

The processor will be in an idle state until it completes tick 4, then register IR will take in a new instruction.

### add Rx, Ry

Suppose Rx contained the value of 23 and Ry contained the value of 11. The instruction add Rx, Ry will result in the following:

Tick 1:

DIN will be read into the instruction register so that the control unit can decode and process the instruction.

Tick 2:

The value from Rx (23) is placed onto the bus. This value is stored into register A.

Tick 3:

The value from Ry (11) is placed onto the bus. This value will be used within the ALU to perform an operation between register A and the value on the bus (Ry). The operation performed will be an addition. Register G will store the results (23+11=34) from the ALU.

Tick 4:

The value within register G is placed onto the bus. This value (34) will be stored in Rx.

### addi Rx Immi

Suppose Rx contained the value of 10 and the immediate value was 15. The breakdown of addi Rx results in the following:

Tick 1:

DIN will be read into the instruction register so that the control unit can decode and process the instruction.

Tick 2:

The immediate value from DIN (15) is placed onto the bus. This value is stored into register A.

Tick 3:

The value from Rx (10) is placed onto the bus. This value will be used within the ALU to perform an operation between register A and the value on the bus (Rx). The operation performed will be an addition. Register G will store the results (15+10=25) from the ALU.

Tick 4:

The value (25) within register G is placed onto the bus. This value will be stored in Rx.

### sub Rx, Ry

Suppose Rx contained the value of 23 and Ry contained the value of 11. The breakdown of sub Rx, Ry results in the following:

Tick 1:

DIN will be read into the instruction register so that the control unit can decode and process the instruction.

Tick 2:

The value from Rx (23) is placed onto the bus. This value is stored into register A.

Tick 3:

The value from Ry (11) is placed onto the bus. This value will be used within the ALU to perform an operation between register A and the value on the bus (Ry). The operation performed will be a subtraction. Register G will store the results (23-11=12) from the ALU.

Tick 4:

The value within register G is placed onto the bus. This value (12) will be stored in Rx.

### mul Rx, Ry

Suppose Rx contained the value of 4 and Ry contained the value of 5. The breakdown of mul Rx, Ry results in the following:

DIN will be read into the instruction register so that the control unit can decode and process the instruction.
The value from Rx (4) is placed onto the bus. This value is stored into register A.
The value from Ry (5) is placed onto the bus. This value will be used within the ALU to perform an operation between register A and the value on the bus (Rx). The operation performed will be a multiplication. Register G will store the results (4*5=20) from the ALU.
The value (20) within register G is placed onto the bus. This value will be stored in Rx.

### ssi Rx

(Updated: Old ordering of ticks is completely acceptable to use) Suppose Rx contained the value of 9, and the immediate value was -2. The breakdown of ssi Rx results in the following:

Tick 1:

DIN will be read into the instruction register so that the control unit can decode and process the instruction.

Tick 2:

The immediate value from DIN (-2) is placed on the bus. This is stored into register A.

Tick 3:

The value from Rx (9) is placed onto the bus, and Register G will store the results (9 << (-2), which is 9 >> 2 = 2) from the ALU.

Tick 4:

The value (2) within register G is placed onto the bus. This value will be stored in Rx.

### movi Rx Immi

Suppose Rx contained the value of 10 and the immediate value was 15. The breakdown of movi Rx results in the following:

Tick 1:

DIN will be read into the instruction register so that the control unit can decode and process the instruction.

Tick 2:

The immediate value from DIN (15) is placed onto the bus. This value (15) overwrites the value stored in Rx. The processor will be in an idle state until it completes tick 4, then register IR will take in a new instruction.

Tick 3 & Tick 4:

The processor will be in an idle state until it completes tick 4, then register IR will take in a new instruction.

**Previous Page** **Next Page**

MONASH University

# Field Guide to Processors, Datapaths and Instruction Execution



**AI & Generative AI tools may be used SELECTIVELY within this assessment.**

In this task, AI can be used as specified for one or more parts of the assessment task as described below.
As a reminder, you can use generative artificial intelligence (AI) in order to troubleshoot your solutions, explain concepts and assist you in preparing your reports. Both students in a group must fully understand the work submitted for assessment - if you do not understand your work, you may receive a zero interview score, which will result in a zero for the project.

Any use of generative AI for this purpose must be appropriately acknowledged, if you do not acknowledge your use of AI, it may be submitted as plagiarism under the academic integrity guidelines. You must not use generative artificial intelligence (AI) to generate any HDL for submission.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

1. represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
2. adhere to the guidelines for AI use set for the assessment task.
3. reflect the University's commitment to academic integrity and ethical behaviour.

*Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.*

This lesson introduces some key background information for the project.

# Memory and Branch Instructions

## Memory and Branch Instructions

To extend the x72 architecture to be fully functional, we need to add instruction memory and branch instructions. This is completed in task 4 of the project. By adding instruction memory, we give the processor the ability to load in instructions automatically on the basis of the value held in a "program counter" register. As this part of the architecture is designed to be a challenge, we will be providing a high level overview of the architecture's behaviour instead of exact implementation details.

### Program Counter

In order for the processor to remember where in memory it should read an instruction from, we will need to add a program counter (PC) register to the x72 datapath. This 16-bit register should store the memory address of where the current/next instruction (depending on which tick we are currently in) can be found. The processor should make this register's value available so that the corresponding data from the instruction memory can be inputted back into the processor through 'Din'.

### Instruction Memory

The instruction memory of the x72 architecture is Read-Only (ROM). This can be implemented using an IP-Block. The ROM takes in an address and outputs the data from that address on the next rising edge of its clock.

If we decide to run the instruction memory using a clock speed far greater than the processor, then, from the processor's point of view, the instruction memory will output its data immediately after the address is changed. Hint: looking at the frequency specified in task 4, this is very possible to achieve.

The contents of the instruction memory should be arranged such that all instructions take up two 'words' of memory. This isn't maximally efficient, as type 1 instructions do not require a second word to store the immediate, but this assumption allows your processor to always know where the next instruction will start, which makes things much easier.

From this assumption, an x72 processor needs to increment the program counter (PC) as follows to execute programs sequentially:

- If it is currently executing a type 1 instruction, add 2 to the PC at the end in order to request the next instruction.
- If it is currently executing a type 2 instruction, add 1 to the PC to retrieve the immediate value from memory, then add another 1 at the end to retrieve the next instruction.

## Branch Instructions

A "branch" instruction is a type of instruction that updates the program counter. By altering the program counter in this fashion, we can jump through our instruction memory. To be properly useful, we need to be able to perform a conditional branch, meaning that we only perform the jump if a condition is met, otherwise, we just continue on. When writing programs, we can use branch instructions to implement conditional statements, and from these, loops and functions.

The x72 processor datapath only defines a single type of branch instruction, bez, which is a "Branch if Equal to Zero" instruction. This means that if the value in the specified register equals 0, the instruction will adjust the PC to jump to the specified instruction.

For example, if r0 had the value '0' in it, *bez r0, 5* would increment the PC by 9, taking it **five** instructions ahead (+ 1 for the current immediate to the next instruction, and +8 for four instructions worth of memory).

Previous Page End Lesson